



Islamic University of Technology

Department of Computer Science and Engineering (CSE)

B.Sc. in Software Engineering

SWE 4404: Software Project Lab II

Summer, 2025–2026

Project Report

QuickPlot

Group 09

Tiran Abdullah	ID 230042105
Md. Ashraful Abedin	ID 230042111
Arafat Hosen	ID 230042151

Supervisor

Samnun Azfar
Junior Lecturer, CSE

Date: May 10, 2026

Contents

1	Project Overview	2
2	Motivation Behind the Project	3
3	User Requirement Analysis	4
3.1	Stakeholder Identification	4
3.2	Functional Requirements	4
3.3	Non-Functional Requirements	6
4	Flowchart	7
5	Key Features	9
5.1	Core Features	9
5.2	Extended Features	10
6	Tools and Technology	11
7	Proposed Timeline	12
8	Suggestions Received	12
9	Links	13

QuickPlot

1 Project Overview

QuickPlot is a browser-based mathematical graphing platform that lets users render and explore mathematical equations interactively — no installation required. It accepts typed equations, parses them in real time, and plots smooth curves on Canvas across 2D Cartesian, 2D polar, and 3D coordinate systems.

The core philosophy is to remove friction from mathematical exploration. Students, researchers, and developers should all be able to visualise a graph within seconds, directly in the browser. Workspaces can be exported and shared via URL, making QuickPlot useful for both individual study and live classroom demonstrations.

Key characteristics of QuickPlot include:

- **Zero-install, browser-first:** built with TypeScript, React, and HTML5 Canvas — runs on any modern browser.
- **Multi-mode rendering:** supports 2D/3D Cartesian, polar, and implicit equations.
- **Parametric sliders:** variable parameters can be bound to draggable sliders for real-time graph exploration.
- **Save & Share:** workspaces serialise to JSON and are shareable through short links.
- **Browser Extension:** a Chrome MV3 side-panel extension eliminates context-switching for online learners.

2 Motivation Behind the Project

The motivation for QuickPlot emerged from a problem observed repeatedly in academic and professional settings: visualising a mathematical equation requires either a heavy desktop application, or navigating away from the current page to an online tool. Neither option is fast or frictionless enough for the rapid, exploratory nature of real mathematical thinking. Some specific points guided the project direction.

The first is a learning gap. Students frequently need to visualise equations from textbooks or lecture slides, but most cannot afford or are not familiar with professional graphing software. This leaves them unable to gain intuitive insight from a formula alone, turning what should be a quick visual check into a significant obstacle.

The second is context switching. When studying online, a learner reading a PDF or a webpage must leave that page, open a separate tool, re-type the equation, and navigate back. Each of these steps interrupts concentration and discourages the exploratory thinking that deepens mathematical understanding. A browser extension embedded into any webpage eliminates this entirely.

The third is the collaboration barrier. Sharing a graph means exporting a static image and attaching it, losing all interactivity. QuickPlot's shareable workspace links allow teachers and students to exchange live, parameterised graphs that the recipient can immediately explore.

Finally, there is the dimension of skill building. Developing a rendering engine, an expression parser, and a 3D graphics pipeline from first principles gives the team practical experience with real-world engineering challenges.

3 User Requirement Analysis

Requirement analysis was conducted through structured discussions with real stakeholders from the intended user groups. The following subsection first identifies the stakeholders and then lists the derived requirements.

3.1 Stakeholder Identification

Table 1: Identified Stakeholders

Stakeholder	Role / Context	Primary Need
Students	Studying Trigonometry, Algebra, Calculus, Signals & Systems, Electromagnetics	Instant graph of textbook equations without leaving study materials
Instructors	Deliver lecture demonstrations; create visual aids	Interactive sliders for live parameter manipulation during class
Researchers	Visualise simulation outputs and model curves	Quick plot from imported dataset; easy result sharing with supervisors
Software Developers	Test mathematical utility functions; explore function behaviour	Fast 2D/3D render; edge-case visualisation

3.2 Functional Requirements

1. Equation Input and Parsing

The system shall accept mathematical expressions typed in standard notation and parse them without requiring the user to learn a special syntax.

2. 2D Cartesian Graphing

The system shall plot one or more functions on a shared Cartesian canvas with a visible, labelled axis grid. Multiple functions shall be distinguished by automatically assigned distinct colours.

3. **2D Polar Graphing**

The system shall support polar equations of the form $r = f(\theta)$ and render the resulting curves on a polar grid overlay.

4. **3D Surface and Parametric Graphing**

The system shall render 3D graphs of the form using Three.js, with mouse-driven orbit and zoom controls for full spatial exploration.

5. **Implicit Equation Rendering**

The system shall support equations of the form $f(x, y) = 0$ by applying an interpolation algorithm to identify and draw the curve.

6. **Parameter Sliders**

Users shall be able to define named constants within an equation and attach interactive sliders to each constant. Adjusting a slider shall trigger an immediate re-render of all affected equations.

7. **Zoom and Pan**

The canvas shall support smooth scroll-to-zoom and click-drag panning so that users can inspect fine detail or view the global shape of a function.

8. **Save and Share Workspace**

The system shall serialise the current set of equations, slider values, and viewport state to a JSON document that can be exported as a .json file or encoded into a shareable URL.

9. **Browser Extension**

A Chrome extension shall embed a QuickPlot panel accessible from any webpage tab, allowing users to graph equations without navigating away from their current page.

10. **Theme Support**

The system shall offer Dark Mode, Light Mode, and some themes.

3.3 Non-Functional Requirements

1. **Performance:** A simple 2D graph shall render the user finishing input on a standard consumer laptop.
2. **Responsiveness:** The UI shall adapt to screen widths without horizontal scroll.
3. **Accessibility:** Colour choices shall meet contrast ratios; keyboard navigation shall be supported throughout.
4. **Security:** User passwords shall never be stored in plain text; all traffic shall be served over HTTPS.
5. **Scalability:** The stateless frontend deployment on Vercel shall handle concurrent users without backend changes.

4 Flowchart

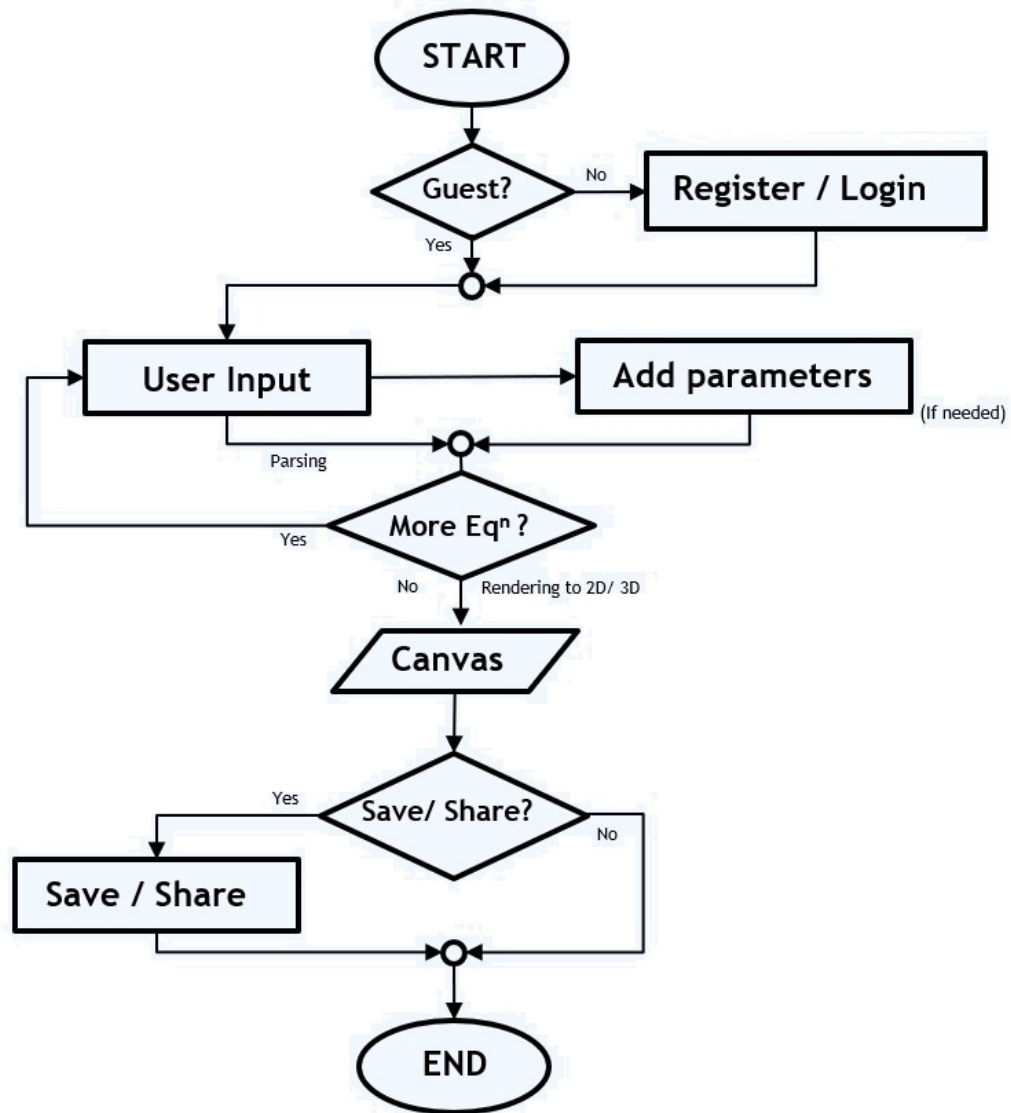


Figure 1: Application Flow Chart — QuickPlot

Flow Description

Start: The application loads in the browser and initialises the canvas, the equation parser.

Check Guest or Authenticated Account A user may proceed immediately as a guest (no login required) or choose to register/log in to unlock workspace persistence and sharing history.

User Input: The user types a mathematical equation into the input panel. The parser validates the expression on every keystroke and reports syntax errors inline.

Add Parameters: If the equation contains unbound constants, the system prompts the user to define slider ranges for each constant.

Parsing: The validated expression is compiled into an evaluable function using the internal expression parser. Both explicit and implicit forms are handled.

Add More Equations: The user may add further equations to overlay multiple plots on the same canvas. Each equation is assigned a unique colour automatically.

Rendering to 2D or 3D: The compiled function(s) are evaluated at uniform sample points across the viewport and rendered onto the HTML5 Canvas (2D) or the Three.js WebGL context (3D).

Canvas: The resulting interactive graph is displayed. The user can zoom, pan, and drag sliders to explore the function dynamically.

Save or Share The user may export the workspace as a JSON file or generate a shareable URL that encodes the full session state. If the user declines, the session remains browser-local.

End: The session concludes when the user closes or navigates away from the application. Authenticated users retain their workspace on the server.

5 Key Features

5.1 Core Features

- **Equation Grapher**

Users can type one or more mathematical functions and instantly see them rendered on a labelled Cartesian grid. Multiple equations can be layered on the same canvas, each assigned a distinct colour automatically.

- **Parameter Sliders**

Named constants within an equation can be bound to interactive sliders. Adjusting a slider triggers an immediate re-render, allowing users to observe how changes in parameters affect the shape of the graph in real time.

- **Zoom & Pan**

The canvas supports smooth scroll-to-zoom and click-drag panning, enabling users to inspect fine detail or step back to view the global behaviour of a function without re-entering any input.

- **Implicit Equation Rendering**

Equations that cannot be expressed as explicit functions are supported through the Marching Squares interpolation algorithm, which traces the curve across the canvas grid and draws it accurately.

- **Polar Graphing**

Users can switch to polar mode and plot equations in polar form. The canvas renders a polar grid with clearly annotated radial and angular axes.

- **Save & Share**

The current workspace, including all equations and slider configurations, can be exported as a JSON file or encoded into a shareable URL that restores the full session state for any recipient.

5.2 Extended Features

- **3D Visualisation Engine**

Users can plot three-dimensional surfaces using WebGL via Three.js. The rendered surface supports mouse-driven orbit and zoom controls for full spatial exploration.

- **Data Visualisation**

Users can import a CSV or JSON dataset and have it rendered as a scatter plot or line chart. A best-fit curve can be overlaid through polynomial interpolation for comparison with theoretical models.

- **User Authentication and Workspace Persistence**

Registered users can save named workspaces to the cloud and retrieve them across sessions. Passwords are securely hashed through Supabase Auth and no plain-text credentials are stored.

- **Browser Extension**

A Chrome Manifest V3 extension embeds QuickPlot as a side panel accessible from any browser tab, allowing users to graph equations without leaving their current page.

- **Theme Support**

The platform offers Dark Mode, Light Mode, and additional featured colour themes. Theme changes apply instantly across the canvas, axes, and UI panels.

- **Responsive Adaptive Layout**

The interface adapts to any screen size, from mobile phones to widescreen desktops, maintaining full usability without horizontal scrolling or layout breakage.

6 Tools and Technology

The following table summarises the technology stack:

Table 2: Tools and Technologies

Category	Tool / Technology	Purpose
Core Language	TypeScript, JavaScript	Type-safe application logic and parser implementation
Frontend Framework	React	Component-based UI; reactive state management
Styling	Tailwind CSS	Utility-first responsive styling
2D Rendering	HTML5 Canvas	High-performance pixel-level 2D graph rendering
3D Rendering	Three.js	WebGL-accelerated 3D surface and parametric plots
Scale / Axes	D3-scale	Domain-to-range mapping for axis tick generation
Backend	Supabase	Auth, real-time database, and REST API
Database	PostgreSQL (via Supabase)	Persistent storage for user accounts and workspaces
Hosting	Vercel	Serverless edge deployment of the React frontend
Browser Extension	Chrome Manifest V3	Side-panel extension for in-page graph access
Version Control	Git / GitHub	Source control, branching, and code review
IDE	Visual Studio Code	Development environment

The architecture is deliberately frontend-heavy: the equation parser, the 2D renderer, and the 3D engine all run entirely in the browser. The backend is used exclusively for authentication and workspace persistence, so the core graphing functionality works even without a network connection once the page has loaded.

7 Proposed Timeline

The Gantt chart below shows the planned schedule:

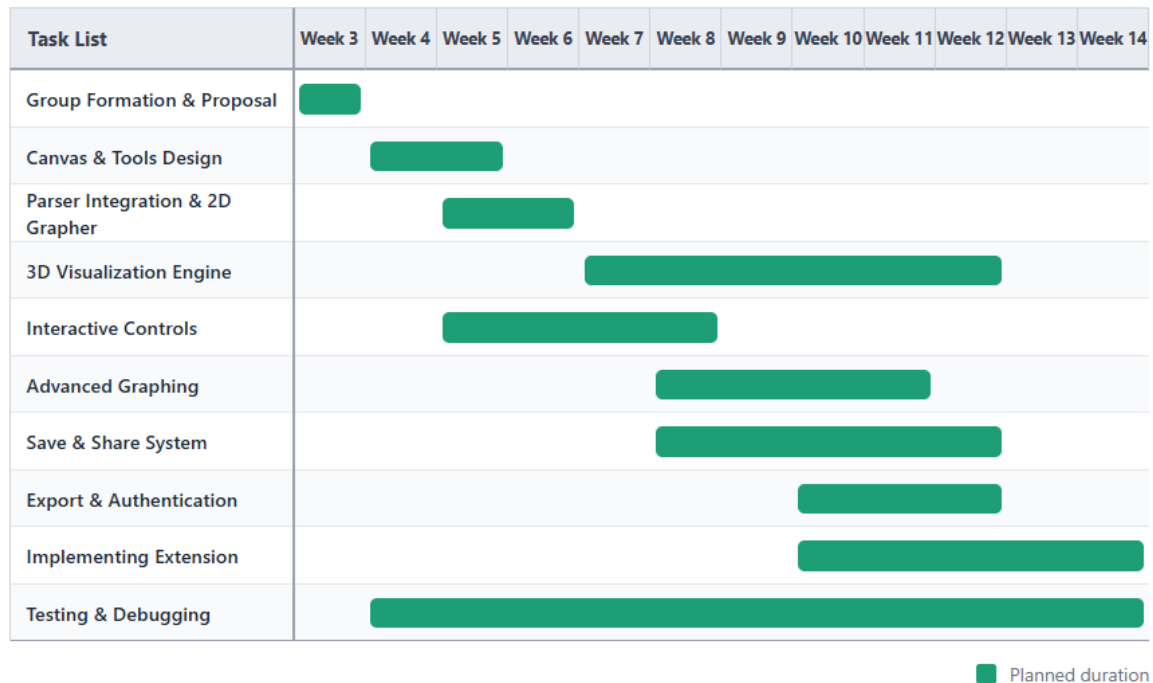


Figure 2: Project Timeline (Gantt Chart)

8 Suggestions Received

The suggestions that we received are -

1. Adding a feature to animate through graph on the canvas

Response: This suggestion has been accepted. By using an external API, it is doable.

2. Adding the User dashboard

Response: This suggestion has already been encountered and also added to the main feature.

If User login, they can see their saved works.

9 Links

1. GitHub Repository — `github.com/AshrafulAbedin/QuickPlot`